

Metodologie di Programmazione (A-L)

II semestre - a.a. 2025 – 2026

Ereditarietà II

a cura di Massimo La Morgia



SAPIENZA
UNIVERSITÀ DI ROMA

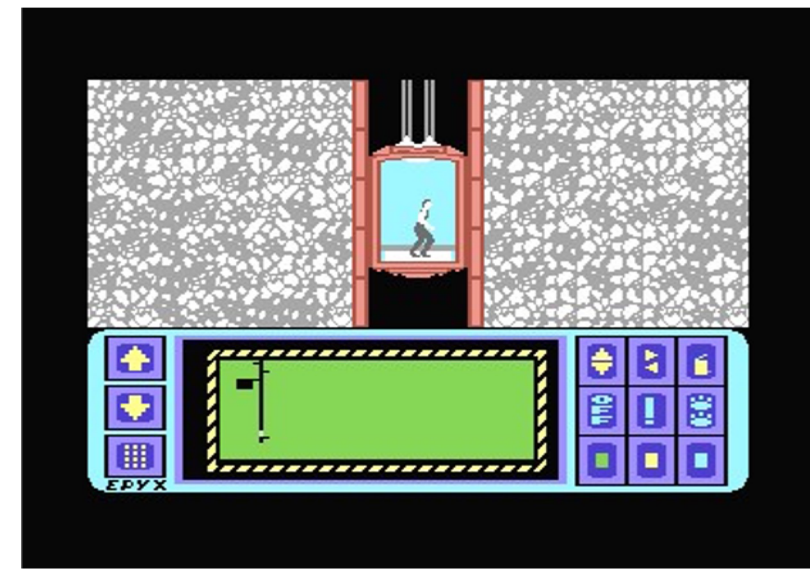
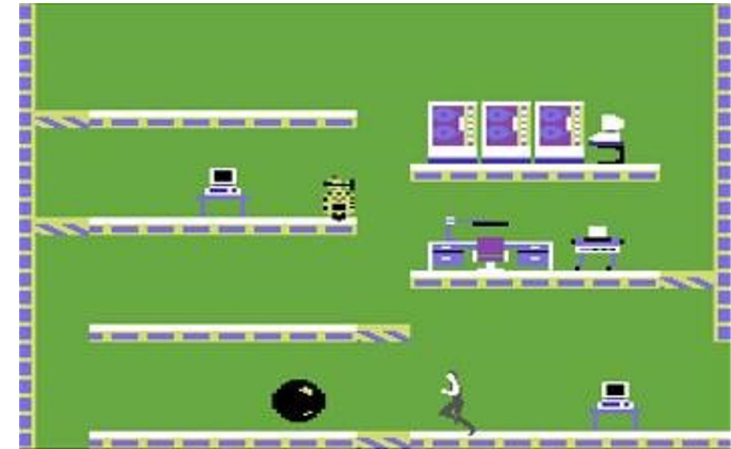
*Tutti i diritti relativi al presente materiale didattico ed al suo contenuto sono riservati a Sapienza e ai suoi autori (o docenti che lo hanno prodotto). È consentito l'uso personale dello stesso da parte dello studente a fini di studio. Ne è vietata nel modo più assoluto la diffusione, duplicazione, cessione, trasmissione, distribuzione a terzi o al pubblico pena le sanzioni applicabili per legge.

**I crediti sulle slide di questo corso sono riportati nell'ultima slide

Ereditarietà parte II

Esempio: Impossible Mission

- Abbiamo tanti "oggetti" (entità) in senso lato:
 - Piattaforme
 - Computer
 - Oggetti in cui cercare indizi
- Alcuni sono "personaggi":
 - Il giocatore
 - I robot
 - Il "bombone"
- Che cosa hanno in comune tutti?
- E che cosa li distingue?



Modellare Impossible Mission in un quarto d'ora: la classe "astratta" Entità

- Abbiamo bisogno di una classe **molto generale** (quindi **astratta**) che rappresenti oggetti mobili e immobili nel gioco:

```
package it.uniroma1.impmis;  
  
abstract public class Entita  
{  
    protected int x;  
    protected int y;  
  
    public Entita(int x, int y)  
    {  
        this.x = x;  
        this.y = y;  
    }  
  
    public int getX() { return x; }  
    public int getY() { return y; }  
}
```

Campi a visibilità protetta

- La visibilità protetta (**protected**) rende visibile il campo (o il metodo) a tutte le sottoclassi
 - Ma anche a **tutte le classi del package** (!)

Modellare Impossible Mission in un quarto d'ora: gli oggetti immobili

- Modelliamo gli **oggetti immobili** in astratto:

```
package it.uniroma1.impmis;  
  
abstract public class Oggetto extends Entita  
{  
    private TesseraPuzzle tessera;  
  
    public Oggetto(int x, int y)  
    {  
        this(x, y, null);  
    }  
  
    public Oggetto(int x, int y, TesseraPuzzle tessera)  
    {  
        super(x, y);  
        this.tessera = tessera;  
    }  
  
    public TesseraPuzzle search() { return tessera; }  
}
```

Doppio costruttore (overloading)

Riuso del codice chiamando l'altro costruttore mediante la parola chiave **this**

Richiama il costruttore della superclasse (**OBBLIGATORIO** perché ha almeno un parametro) con la parola chiave **super**

Metodo aggiuntivo

Modellare Impossible Mission in un quarto d'ora: gli oggetti immobili

- che possono contenere una tessera del puzzle del gioco:

```
package it.uniroma1.impmiss;
```

```
public class TesseraPuzzle  
{  
    // da implementare  
}
```

Modellare Impossible Mission in un quarto d'ora:

un esempio di oggetto e il computer

- Modelliamo un possibile oggetto immobile:

```
package it.uniroma1.impmiss;

public class Libreria extends Oggetto
{
    public Libreria(int x, int y, TesseraPuzzle tessera)
    {
        super(x, y, tessera);
    }

    public Libreria(int x, int y)
    {
        super(x, y);
    }
}
```



```
package it.uniroma1.impmiss;

abstract public class Entita
{
    protected int x;
    protected int y;

    public Entita(int x, int y)
    {
        this.x = x;
        this.y = y;
    }

    public int getX() { return x; }
    public int getY() { return y; }
}
```

- e il computer:



```
package it.uniroma1.impmiss;

public class Computer extends Entita
{
    public Computer(int x, int y)
    {
        super(x, y);
    }

    public void login() { /* da implementare */ }
    public void logout() { /* da implementare */ }
}
```

Metodi aggiuntivi

```
package it.uniroma1.impmiss;

abstract public class Oggetto extends Entita
{
    private TesseraPuzzle tessera;

    public Oggetto(int x, int y)
    {
        this(x, y, null);
    }

    public Oggetto(int x, int y, TesseraPuzzle tessera)
    {
        super(x, y);
        this.tessera = tessera;
    }

    public TesseraPuzzle search() { return tessera; }
}
```

Modellare Impossible Mission in un quarto d'ora:

entità mobili

- Modelliamo un generico personaggio:

```
package it.uniroma1.impmis;  
  
abstract public class Personaggio extends Entita  
{  
    public enum Direzione  
    {  
        DESTRA,  
        SINISTRA,  
        ALTO,  
        BASSO;  
    }  
  
    private String nome;  
    private int velocita;  
  
    public Personaggio(int x, int y, String nome, int velocita)  
    {  
        super(x, y);  
        this.nome = nome;  
        this.velocita = velocita;  
    }  
  
    public String getNome() { return nome; }  
    public int getVelocita() { return velocita; }  
  
    public void muoviti(Direzione d)  
    {  
        switch(d)  
        {  
            case DESTRA: x += velocita; break;  
            case SINISTRA: x -= velocita; break;  
            // in futuro: emetti eccezione!  
            default: System.out.println("Direzione non ammessa"); break;  
        }  
    }  
}
```

Enumerazione
delle direzioni

Campi aggiuntivi

Si possono fornire
implementazioni
nella classe astratta!

Metodi aggiuntivi

Modellare Impossible Mission in un quarto d'ora: il giocatore

- Modelliamo il `giocatore` (ovvero la spia):

```
package it.uniroma1.impmis;  
  
public class Spia extends Personaggio  
{  
    public Spia(int x, int y, String nome, int velocita)  
    {  
        super(x, y, nome, velocita);  
    }  
  
    public void salta()  
    {  
        // ...  
    }  
}
```



Modellare Impossible Mission in un quarto d'ora:

- Modelliamo un generico nemico:

```
package it.uniroma1.impmiss;  
  
abstract public class Nemico extends Personaggio  
{  
    public Nemico(int x, int y, String nome, int velocita)  
    {  
        super(x, y, nome, velocita);  
    }  
  
    abstract public void attacca();  
}
```

Metodo astratto

- E i robot



```
package it.uniroma1.impmiss;  
  
public class Robot extends Nemico  
{  
    public Robot(int x, int y, String nome, int velocita)  
    {  
        super(x, y, nome, velocita);  
    }  
  
    public void incenerisci() { /* fulmine elettrico */ }  
  
    public void attacca() { /* da implementare */ }  
}
```

Nella sottoclasse siamo obbligati a definire il metodo astratto

Modellare Impossible Mission in un quarto d'ora:

- Modelliamo un **bombone**:



```
package it.uniroma1.impmis;  
  
public class Bombone extends Nemico  
{  
    public Bombone(int x, int y, String nome, int velocita)  
    {  
        super(x, y, nome, velocita);  
    }  
  
    public void attacca()  
    {  
        // da implementare  
    }  
  
    public void muoviti(Direzione d)  
    {  
        switch(d)  
        {  
            case DESTRA: case SINISTRA: super.muoviti(d); break;  
            case ALTO: y -= getVelocita(); break;  
            case BASSO: y += getVelocita(); break;  
        }  
    }  
  
    public void muoviti(Spia p)  
    {  
        muoviti(p.x > this.x ? Direzione.DESTRA : Direzione.SINISTRA);  
        muoviti(p.y > this.y ? Direzione.ALTO : Direzione.BASSO);  
    }  
}
```

Overriding del
metodo

Overloading del
metodo

Esempi di riuso
del codice

Esercizio:

Disegnare il diagramma delle classi di Impossible Mission.

Potete utilizzare qualsiasi strumento di disegno, anche un power point, o (meglio) l'estensione di Google Workspace:
<https://www.drawio.com/>

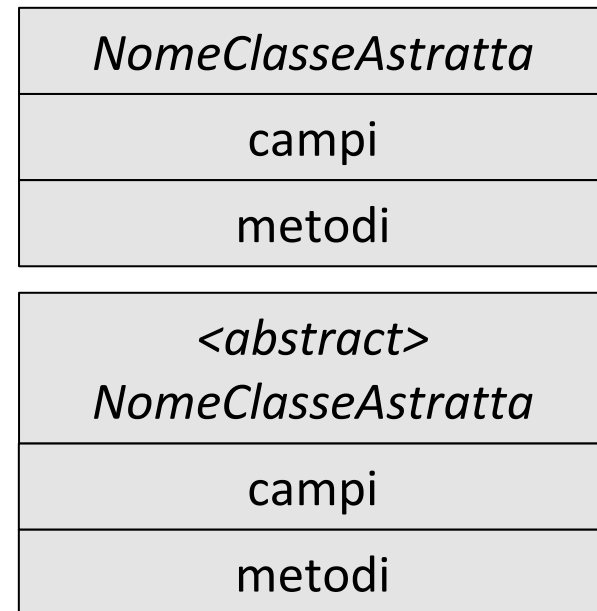
Esercizio:

Disegnate il diagramma delle classi risultante: Entita, Oggetto, TesseraPuzzle, Libreria, Computer, Personaggio, Spia, Nemico, Robot, Bombone

Modellare Impossible Mission in un quarto d'ora:

Disegnate il diagramma delle classi risultante: Entita, Oggetto, TesseraPuzzle, Libreria, Computer, Personaggio, Spia, Nemico, Robot, Bombone

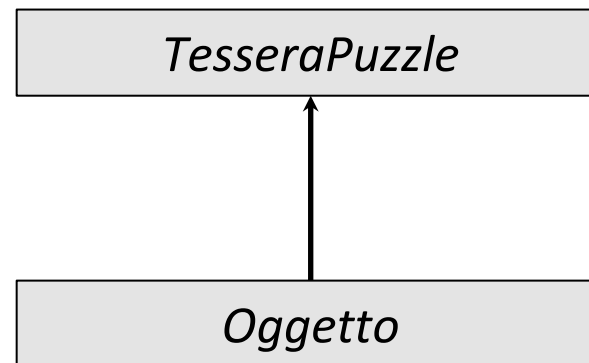
In UML una classe astratta ha il nome in corsivo e/o si usa il tag <abstract>



Modellare Impossible Mission in un quarto d'ora:

Disegnate il diagramma delle classi risultante: Entita, Oggetto, TesseraPuzzle, Libreria, Computer, Personaggio, Spia, Nemico, Robot, Bombone

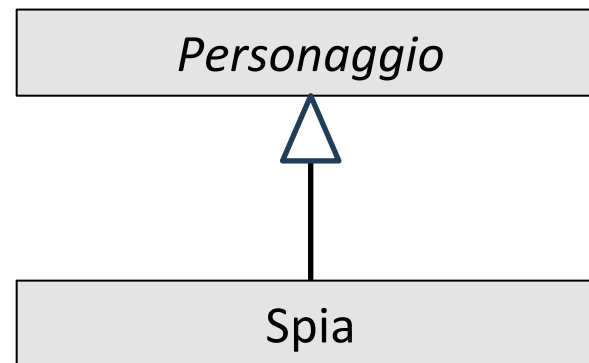
In UML una dipendenza semplice si indica con una freccia



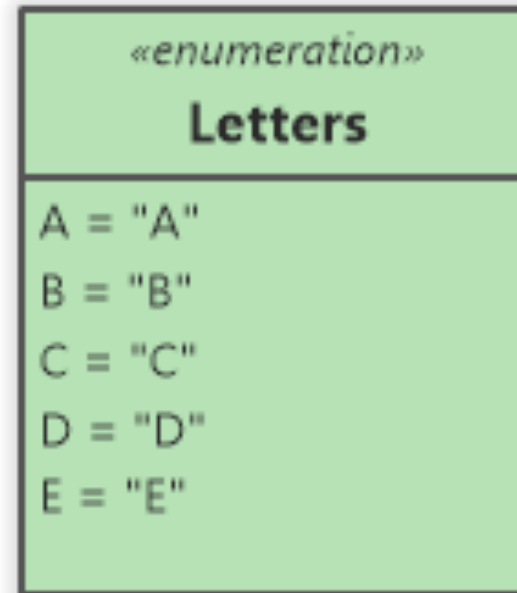
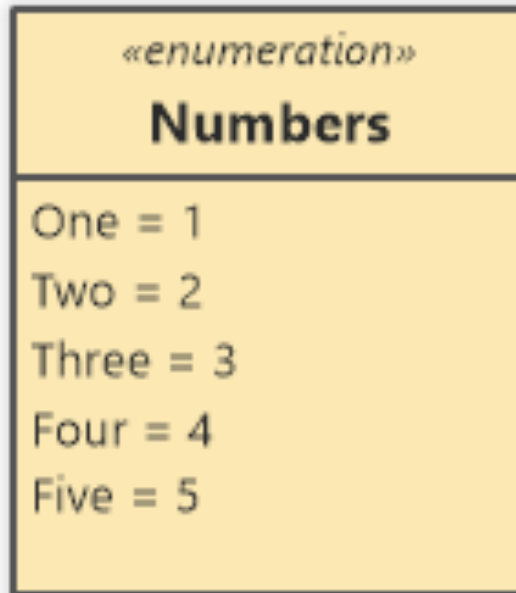
Modellare Impossible Mission in un quarto d'ora:

Disegnate il diagramma delle classi risultante: Entita, Oggetto, TesseraPuzzle, Libreria, Computer, Personaggio, Spia, Nemico, Robot, Bombone

In UML una relazione di estensione



Enumerazioni in UML



Differenza tra Overriding e Overloading

- L'**overriding** consiste nel **ridefinire** (**reimplementare**) un metodo con la stessa **intestazione** ("segnatura") presente in una superclasse
- L'**overloading** consiste nel creare un metodo con lo stesso nome, ma una **intestazione** diversa (diverso numero e/o tipo di parametri)

Mantenere il contratto

- Nell'**overriding** gli argomenti devono essere gli stessi
- I tipi di ritorno devono essere compatibili (lo stesso tipo o una sottoclasse)
- Non si può **ridurre la visibilità** (es. da public a private)
- Nell'**overloading** i tipi di ritorno possono essere diversi
 - **MA:** non si può cambiare **SOLO** il tipo di ritorno
- Si può **variare la visibilità** in qualsiasi direzione

Visibilità

Quattro possibilità per campi e metodi:

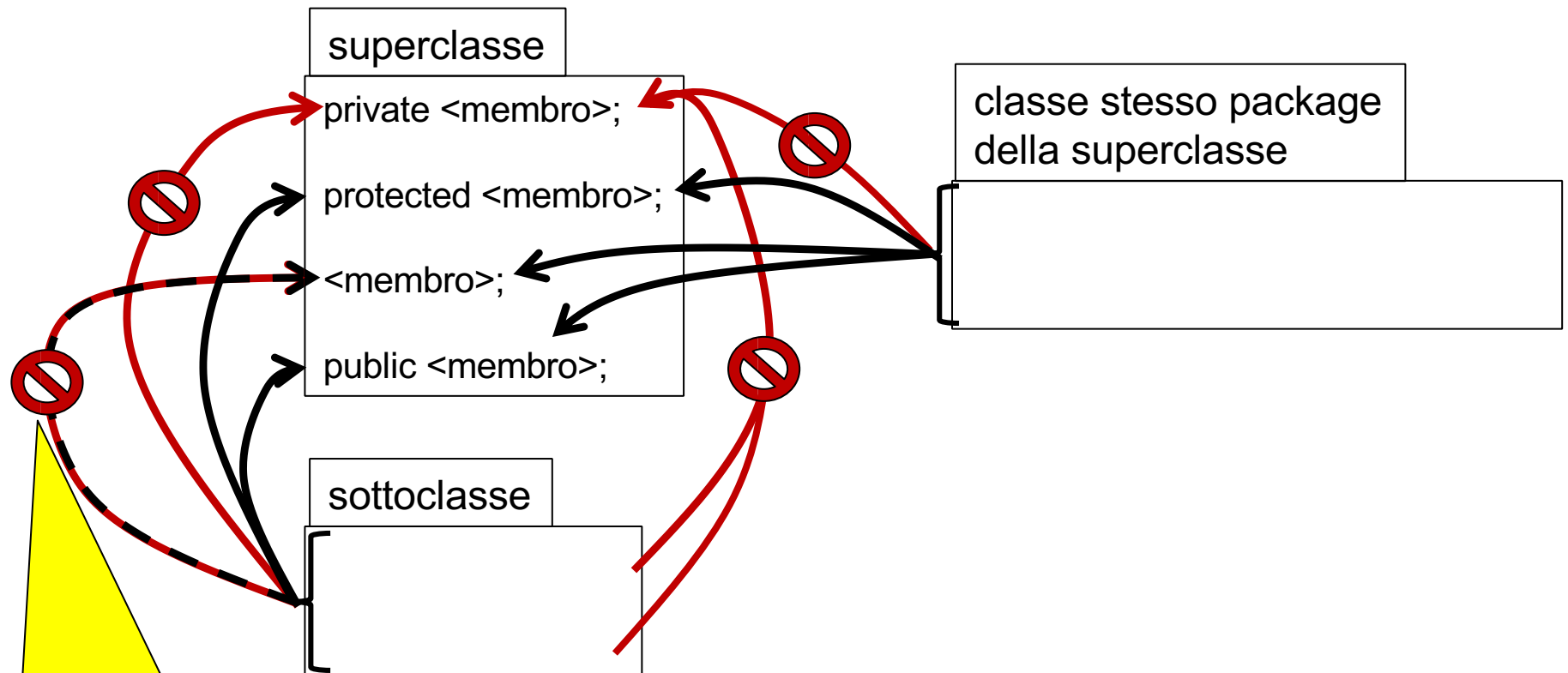
- **Private**: visibile solo all'interno della classe
- **Public**: visibile a tutti [all'interno di un modulo, vedi Java >=9]
- **Default**: visibile all'interno di tutte le classi del package
- **Protected**: visibile all'interno di tutte le classi del package e delle sottoclassi (indipendentemente dal package)

	Class	Package	Subclass (same pkg)	Subclass (diff pkg)	World
<code>public</code>	✓	✓	✓	✓	✓
<code>protected</code>	✓	✓	✓	✓	✗
<i>no modifier</i>	✓	✓	✓	✗	✗
<code>private</code>	✓	✗	✗	✗	✗

✓: accessible

✗: not accessible

Visibilità



Accessibile se la sottoclasse
è nello stesso package

Is-a contro has-a



- E' molto importante distinguere tra relazioni di tipo è-un (is-a) e relazioni di tipo ha-un (has-a)
- Is-a rappresenta l'ereditarietà

Un oggetto di una sottoclasse può essere trattato come un oggetto della superclasse

Domanda: la sottoclasse è-un superclasse? (es. Paperino è un PersonaggioDisney? Sì! QuiQuoQua è un Paperino? No!)

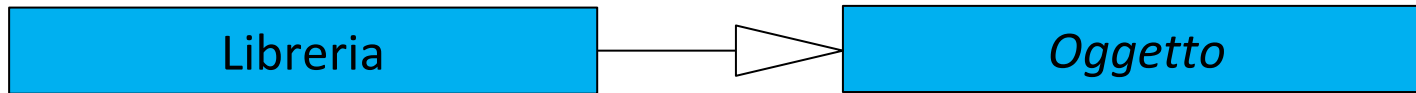
- Has-a rappresenta la composizione

Un oggetto contiene come membri riferimenti ad altri oggetti

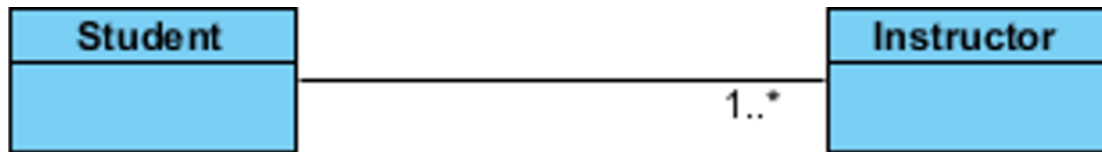
Domanda: un oggetto contiene altri oggetti? (es. Bagno contiene Vasca? Sì! PersonaggioDisney contiene Paperino? No!)

In UML

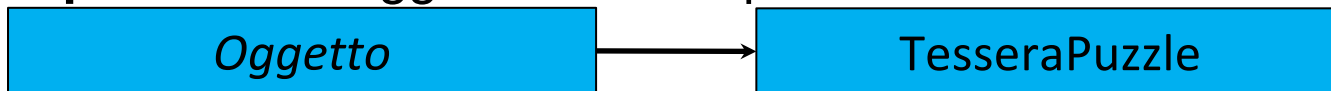
Is-A: implica una relazione gerarchica di ereditarietà



Associazione: implica una relazione generica che associa x oggetti di una classe a y oggetti di un'altra classe

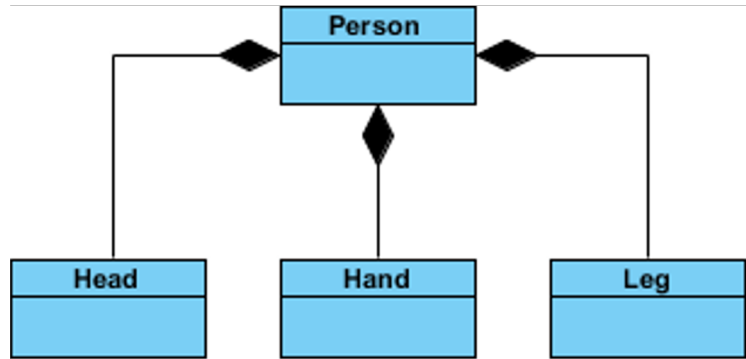


Dipendenza: Oggetto fa uso/dipende di/a TesseraPuzzle

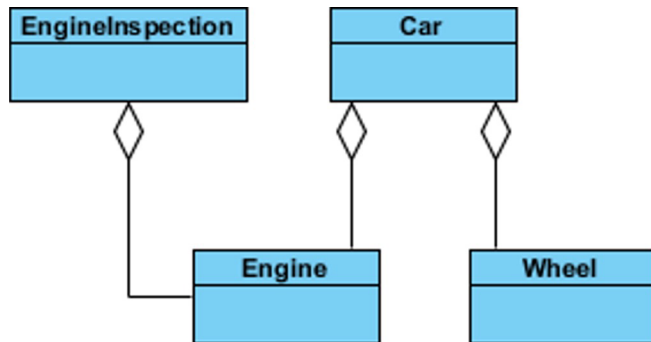


In UML

Composizione: implica una relazione dove un oggetto non può esistere indipendentemente da un altro (ad es. Casa e Stanza)



Aggregazione: implica una relazione dove un oggetto può esistere indipendentemente dall'altro.



Esercizio: BarraDiEnergia & BarraDiEnergiaConPercentuale

- Creare una classe `BarraDiEnergia` costruita con un intero che ne determina la lunghezza massima. Inizialmente la barra è vuota. La barra è dotata di un metodo per l'incremento unitario del suo livello di riempimento e di un metodo `toString` che ne fornisca la rappresentazione sotto forma di stringa (es. se il livello è 3 su 10, la stringa sarà "OOO=====").
- Creare quindi una seconda classe `BarraDiEnergiaConPercentuale` che fornisce una rappresentazione sotto forma di stringa come `BarraDiEnergia` ma stampando in coda alla stringa la percentuale del livello di riempimento. Per esempio, se il livello è 3 su 10, la stringa sarà "OOO===== 30%".

Esercizio: ListaDiInteri e ListaOrdinataDiInteri

Implementare una classe `ListaDiInteri` mediante un array (con i metodi specificati in fondo alle diapositive della terza parte: "controllo e array")

- Implementare quindi una classe `ListaOrdinataDiInteri` per creare liste di interi ordinati in modo crescente. La classe ridefinisce i seguenti 3 metodi di aggiunta:
 - `aggiungiInCoda(k)`: Aggiungi un intero in coda alla lista: l'aggiunta avviene solo se l'intero preserva l'ordine della lista.
 - `aggiungi(k, j)`: Aggiungi un intero nella posizione specificata: come sopra, l'aggiunta avviene solo se l'intero preserva l'ordine degli interi della lista
 - `aggiungi(k)`: Aggiungi un intero: l'intero viene inserito nella posizione appropriata, in modo da preservare l'ordine degli interi della lista
- L'array non deve essere ordinato con metodi di sorting, quali `Arrays.sort` (né vostri metodi di sorting *completo* dell'array)
- **Extra:** permettere di specificare un parametro da passare opzionalmente al costruttore di `ListaOrdinataDiInteri` per stabilire l'ordine della lista (crescente o decrescente; per default, crescente)

Esercizio: Animali

- Progettare (diagramma delle classi) ed implementare la classe `Animale` che rappresenti un generico animale
- La classe possiede i metodi `emettiVerso()` e `getNumeroDiZampe()`
- Possiede inoltre il metodo `getTaglia()` che restituisce un valore scelto tra: piccola, media e grande.
- Progettare (diagramma delle classi) ed implementare quindi le classi `Mammifero`, `Felino`, `Gatto` (taglia piccola), `Tigre` (grande), `Cane`, `Chihuahua` (piccola), `Beagle` (media), `Terranova` (grande), `Uccello`, `Corvo` (media), `Passero` (piccola), `Millepiedi` (piccola)
- Personalizzare in modo appropriato la taglia, il numero di zampe e il verso degli animali

Esercizio: Conto bancario

- Progettare la classe `ContoBancario` che rappresenti un conto con informazioni relative al denaro attualmente disponibile, il codice IBAN
- Modellare quindi una generica operazione bancaria `Operazione` che disponga di un metodo `esegui()`
- Modellare quindi i seguenti tipi di operazione:
 - `PrelevaDenaro`: preleva una specificata quantità di denaro da un dato conto
 - `SvuotaConto`: preleva tutto il denaro da un dato conto
 - `VersaDenaro`: versa del denaro sul conto specificato
 - `SituazioneConto`: stampa l'attuale saldo del conto
 - `Bonifico`: preleva del denaro da un conto e lo versa su un altro
- Specificare un metodo nella classe `ContoBancario` che restituisca l'elenco delle operazioni svolte in ordine temporale

Esercizio: Distributore automatico

- Progettare una classe `Prodotto` con un prezzo e tre tipi diversi di prodotto: `BottigliaDAcqua`, `BarraDiCioccolato`, `GommeDaMasticare`
- Progettare la classe `DistributoreAutomatico` che rappresenti un distributore automatico costruito con un intero `N` che determina il numero di prodotti nel distributore
- La classe prevede i seguenti metodi:
 - un metodo `carica()` che inserisce `N` prodotti di tipo e ordine casuale
 - un metodo `inserisciImporto()` che permette di inserire un importo nella macchinetta
 - un metodo `getProdotto()` che, dato in ingresso un numero di prodotto, restituisca il prodotto associato a quel numero e decrementi il saldo disponibile nel distributore
 - Un metodo `getSaldo()` che restituisca il saldo attuale del distributore
 - un metodo `getResto()` che restituisca il resto dovuto e azzeri il saldo

Esercizio: Espressioni matematiche

- Progettare una serie di classi che modellino le espressioni matematiche secondo la seguente definizione:
 - Una **costante** di tipo double è un'espressione
 - Una **variabile** con nome di tipo stringa e valore double è un'espressione
 - Se e_1 è un'espressione, allora $-e_1$ è un'espressione
 - Se e_1, e_2 sono espressioni, allora $e_1 \text{ op } e_2$ è un'espressione dove op può essere l'operatore $+, -, *, /, \%$
- Ogni tipo di espressione (costante, variabile, espressioni composte) deve essere modellata mediante una classe separata
- Ogni espressione dispone del metodo `getValore()` che restituisce il valore che quell'espressione possiede in quel momento
- Costruire quindi l'espressione $-(5+(3/2)-2)*x$ e calcolarne il valore quando la variabile x vale 3 e quando la variabile x vale 6
- **Suggerimenti:** la variabile può modificare il proprio valore nel tempo; servirà veramente salvare un valore nella superclasse?
- **Alternative:** progettarlo usando l'ereditarietà (meglio) e mediante enum per le espressioni binarie

Esercizio: il Gioco dell'Oca

- Progettare il Gioco dell'Oca modellando:
 - Il **Giocatore**, che mantiene l'informazione sulla posizione nel tabellone e i punti accumulati e implementa il metodo **tiraDadi()**
 - Il **Tabellone** come sequenza di caselle costruita a partire da un intero N e da un elenco di giocatori; la classe dispone dell'operazione di posizionamento dei giocatori tenendo conto dell'effetto "gioco dell'oca" in cui, arrivati alla fine, si torna indietro
 - Diversi **tipi di caselle** ciascuna con un diverso effetto a seguito del posizionamento del giocatore su quella casella:
 - una **CasellaVuota** (nessun effetto sul giocatore)
 - una **CasellaSpostaGiocatore** che sposta il giocatore di x caselle (avanti se $x > 0$ o indietro se $x < 0$)
 - una **CasellaPunti** che ha l'effetto di far guadagnare o perdere un certo numero di punti al giocatore
 - la classe **GiocoDellOca** che, dato un intero N e dati i giocatori, che inizializza un tabellone di lunghezza N e implementa il metodo **giocaUnTurno()** che fa effettuare una mossa a ognuno dei giocatori

Esercizio: Tetris

- Progettare il gioco del Tetris modellando la classe **Pezzo** con le seguenti operazioni:
 - **Left** : sposta a sinistra il pezzo
 - **Right**: sposta a destra il pezzo
 - **Rotate**: ruota il pezzo in senso orario
 - **Down**: manda giù il pezzo
- Progettare anche la classe di ciascun pezzo (a forma di L, a forma di T, a serpente, a forma di I e cubo)
- Progettare infine la classe **Tetris** che somministra i pezzi, permette al giocatore di muoverli, gestisce lo spazio libero e calcola i punteggi del giocatore

Credits

Le slide di questo corso sono basate sulle slide del Prof. Faralli che sono a loro volta una rielaborazione delle slide del Prof. Navigli

In aggiunta, le slide sono state revisionate dagli studenti borsisti della Facoltà di Ingegneria Informatica, Informatica e Statistica: Mario Marra e Paolo Straniero.

Font ad alta leggibilità Biancoenero® di biancoenero edizioni srl, disegnata da Umberto Mischi, con la consulenza di Alessandra Finzi, Daniele Zanoni e Luciano Perondi (Brevetto n. RM20110000128).

Disponibile gratuitamente per tutte le istituzioni e i privati che la utilizzino per scopi non commerciali.